ONESIM

Single Order Service

Part 1 of 2 — Order Creation, Provisioning & Delivery

Companion document: **Part 2 — Bulk Group Order Service**

ONETECH PRO LLC | v1.0 | March 2026 | Confidential

1. What This Service Does

The Single Order Service handles eSIM orders that do not belong to a pre-built customer group. A tour operator (partner) opens a New Order modal, selects a package, chooses who it is for, and the service provisions the eSIM(s), generates tokenized delivery links, and dispatches notifications — all within a single modal interaction.

This is Part 1 of the ONESIM B2B ordering system. It handles the three lightweight order modes. Part 2 (Bulk Group Order Service) handles the pre-created group flow, which is its own more complex pipeline.

How this document relates to Part 2

Mode 3 (Guruh uchun) in this modal is an ENTRY POINT into the Part 2 pipeline — not a separate implementation.

When the partner selects 'Guruh uchun', the modal collects the group selection and delegates entirely to the Bulk Order Service.

Modes 1 and 2 are fully defined here — they have their own lightweight pipeline that does not use group_orders.

2. The Three Order Modes

The New Order modal presents the partner with three mutually exclusive modes. The entire form — its fields, validation rules, and downstream pipeline — changes based on which mode is selected.

MODE 1	MODE 2	MODE 3
Tur agent nomiga	**Mijoz uchun**	**Guruh uchun**
For the agent themselves	*For named client(s)*	*For a pre-built group*
• eSIM belongs to the partner	• 1 or more named customers	• Selects existing group
• Phone autofilled from profile	• Added inline in the modal	• Group has its own members
• Quantity: 1 or more	• Phone / email per customer	• Full bulk pipeline triggered
• Can share link manually	• Delivery per customer	• Delegates to Part 2
→ Pipeline: Quick Order	**→ Pipeline: Client Order**	**→ Delegates: to Part 2 doc**

3. New Order Modal — UX Specification

3.1 Modal Structure (shared across all modes)

The modal opens when the partner clicks 'Yangi buyurtma' (New Order). It has two phases:

- Phase A — Configuration: package selection, mode selection, mode-specific fields
- Phase B — Result: after successful order, same modal transitions to show eSIM details + confirmation

Package Selection (top of modal, always visible)

This section appears first and is identical for all three modes:

- Country/destination search field — searches esim_packages by location_code or name
- Package list — filtered results showing: name, data amount, validity, price (partner's discounted price)
- Selected package shown as a highlighted card with: flag, country, GB, days, partner price

Mode Selector (tabs or radio segmented control)

Three clearly labeled tabs below the package selection:

Tab Label (UZ)	English meaning	When to use
Tur agent nomiga	In the agent's name	Agent orders for their own device or wants the eSIM link to distribute manually
Mijoz uchun	For a client	Agent knows the customer(s) and wants eSIM delivered directly to them
Guruh uchun	For a group	Agent has a pre-created group and wants to bulk-order for all members at once

3.2 Mode 1: Tur agent nomiga — Form Fields

This is the simplest flow. The eSIM is for the partner themselves (or for distribution — they own the link).

Field	Type	Behaviour
eSIM qayerga yuborilsin?	Phone input	Label: 'eSIM qayerga yuborilsin?' (Where to send the eSIM?). AUTOFILLED with partners.business_phone from the partner's profile. Partner can clear and type a different number. Placeholder: 998 XX XXX XX XX. If the quantity > 1, this field is hidden — multiple eSIMs cannot all go to one phone number as individual tokens.
Nechta eSIM?	Number input	Label: 'Nechta eSIM?' (How many eSIMs?). Default: 1. Min: 1, Max: 30 (eSIMAccess API limit per batch). When value = 1 and delivery is 'SMS': phone field is shown. When value > 1: phone field disappears, delivery method auto-switches to 'manual' — partner downloads QR codes themselves.

Yetkazib berish usuli	Select / auto	When qty = 1: select between SMS or Manual. When qty > 1: locked to Manual. Default when phone present: SMS. Label: 'Yetkazib berish usuli'.

Autofill logic — phone number

Source: partners.business_phone (set during partner registration).

If business_phone is null: field is empty, placeholder shown, partner must fill manually.

Phone format enforced: must start with 998, 12 digits total (no + prefix in DB).

The autofilled phone is for SMS delivery only — it is NOT stored as a 'customer'. The partner owns this order.

3.3 Mode 2: Mijoz uchun — Form Fields

The partner adds 1 or more named customers directly in the modal. Each customer is a row in a mini-table that expands inline.

Field	Type	Behaviour
Customer rows	Dynamic list	Starts with 1 empty row. '+Mijoz qo'shish' button adds another row. Each row is one customer. No upper limit shown (practical limit: 10–15 for this mode — for larger, use Guruh uchun).
Full name	Text input	Per customer. Required. Placeholder: 'Ism Familiya'. Stored in partner_customers if 'Saqlash' is checked.
Phone	Phone input	Per customer. Conditional: required if delivery = SMS. 998XXXXXXXXX format.
Email	Email input	Per customer. Conditional: required if delivery = Email. Standard email validation.
Yetkazib berish	Select	Per customer. Options: SMS, Email, Manual. Default: SMS if phone filled, Email if email filled first, Manual otherwise. Partner can override.
Mijozni saqlash?	Checkbox	Optional per customer. If checked: customer is upserted into partner_customers table. If unchecked: customer data is used for this order only and not persisted. Default: unchecked.

'Saqlash' checkbox — why it matters

When unchecked: customer name/phone/email exists only on the orders row (customer_first_name, customer_email, customer_phone columns). No partner_customers record created.

When checked: partner_customers row is upserted (match on partner_id + email or partner_id + phone). Saved customer appears in the partner's customer list for future reuse.

This is a privacy-by-design choice — not every one-time traveller needs to be stored permanently.

3.4 Mode 3: Guruh uchun — Form Fields

This mode is the entry point to Part 2. The modal is lightweight here — just group selection and confirmation.

Field	Type	Behaviour
Guruh tanlang	Searchable select	Lists partner's customer_groups where status = 'ready'. Shows: group name, member count, destination, travel dates. Partner picks one.
Paket	Auto-filled	Autofills with group's default_package_code if set. Partner can override. Shows package card same as other modes.
Yetkazib berish	Auto-filled	Defaults to group's default_delivery_method. Can be overridden for this order (does not change group default).
Qo'shimcha eslatma	Textarea	Optional note for this order. Stored in group_orders.notes.

When the partner clicks 'Tasdiqlash' (Confirm) in Mode 3, this modal closes and the full Part 2 pipeline is triggered for the selected group. The modal does NOT wait for the bulk pipeline to complete — it shows a progress screen.

4. Pipeline — Mode 1: Tur Agent Nomiga

This is the fastest path. Single (or multiple) eSIMs ordered for the partner themselves. No customer records involved.

4.1 Frontend Behaviour on Submit

#	Step / Action	Detail
F.1	**Validate form**	Check: package selected, quantity ≥ 1. If qty = 1 and delivery = SMS: check phone is valid 998XXXXXXXXX. If qty > 1: no phone needed.
F.2	**Show loading state**	Disable 'Buyurtma berish' button. Replace button text with spinner + 'Yuklanmoqda...' (Loading...). Modal stays open.
F.3	**Call Edge Function**	POST /functions/v1/process-single-order. Payload: { mode: 'agent', package_code, quantity, delivery_method, phone (if SMS, qty=1) }.
F.4	**On success response**	Transition modal to Result Phase (see Section 4.3). Show toast: success message.
F.5	**On error response**	Show error banner inside modal. Button re-enables. Partner can retry.

4.2 Backend: process-single-order (mode: agent)

#	Step / Action	Detail
B.1	**Auth & partner lookup**	Verify JWT. Load partners row for calling user. Confirm partner.status = 'active'.
B.2	**Pre-flight: eSIMAccess balance**	POST /api/v1/open/balance/query. Response obj.balance in units×10000. Calculate order cost: package.api_price × quantity. If balance < order cost: return error 'Hisobda mablag' yetarli emas' (Insufficient balance). Do NOT proceed.
B.3	**Generate transactionId**	UUID v4 prefixed: 'AGENT-{partner_id_short}-{timestamp}'. Max 50 chars. This is the eSIMAccess idempotency key — store it before calling the API.
B.4	**Call eSIMAccess /esim/order**	POST /api/v1/open/esim/order. Body: { transactionId, packageInfoList: [{ packageCode: package.package_code, count: quantity, price: package.api_price }] }. Store returned orderNo immediately.
B.5	**Poll for allocation**	eSIMAccess allocates asynchronously. Poll POST /api/v1/open/esim/query with orderNo every 3 seconds, max 10 attempts (~30s). Wait for esimList[n].esimStatus = 'GOT_RESOURCE' for all N profiles. If not ready after 10 polls: return partial result with orderNo for frontend to show 'processing' state.

B.6	**Generate tokens**	For each provisioned eSIM: generate nanoid(8) token. Verify uniqueness against orders.short_url. Build URL: onesim.uz/e/{token}.
B.7	**Write orders rows**	For each eSIM: INSERT into orders: { user_id = partner.user_id, partner_id, package_code, iccid, qr_code_data = esim.ac (activation code), smdp_address, activation_code, short_url = token URL, order_status = 'ALLOCATED', source_type = 'b2b_partner', delivery_method, discount fields, end_customer_type = 'b2b_partner_customer', customer_phone = provided phone (if mode=agent + SMS) }.
B.8	**Write partner_earnings**	INSERT partner_earnings row per eSIM.
B.9	**Update partner stats**	UPDATE partners: total_orders += quantity, total_spent += partner_paid_total, total_savings += discount_total, last_order_at = now().
B.10	**SMS delivery (if requested)**	If delivery_method = 'sms' AND quantity = 1: call Eskiz. See Section 6 for full Eskiz flow. Store delivery result in esim_delivery_logs.
B.11	**Return response**	Return: { orders: [{ id, iccid, short_url, qr_code_data, activation_code, package_name, delivery_status }], sms_sent: true/false }.

4.3 Modal Result Phase — What the Partner Sees

After successful response, the modal content is replaced (no close/reopen — smooth in-place transition):

- Green checkmark header: 'eSIM muvaffaqiyatli buyurtma qilindi!' (eSIM successfully ordered!)
- If qty = 1: eSIM card showing — package name, country flag, data/validity, ICCID (last 4 digits), QR code image (rendered from activation_code)
- If qty > 1: list of N eSIM rows, each with ICCID suffix and a 'Nusxa olish' (Copy) button for the token URL
- Token URL shown with copy button: 'onesim.uz/e/{token}' — for manual sharing via Telegram/WhatsApp
- If SMS was sent: green info line — 'eSIM +998XXXXXXXXX raqamiga SMS orqali yuborildi' (eSIM sent to +998XXXXXXXXX via SMS)
- 'Yopish' (Close) button and 'Yangi buyurtma' (New order) button

Toast message

Simultaneously with modal transition, a toast notification fires at the top-right of the screen.

SMS delivery: '✓ eSIM tayyor! +998XXXXXXXXX raqamiga SMS yuborildi'

Manual (qty>1): '✓ {N} ta eSIM buyurtma qilindi. Linklar nusxalash uchun tayyor.'

Duration: 5 seconds. Non-blocking.

5. Pipeline — Mode 2: Mijoz Uchun

One or more named customers. Each gets their own eSIM, their own token, and their own delivery. This pipeline runs per-customer sequentially, same as the bulk pipeline but without a group_orders wrapper.

5.1 Frontend Behaviour on Submit

#	Step / Action	Detail
F.1	**Validate all customer rows**	For each customer row: name required. If delivery = SMS: phone required and valid. If delivery = Email: email required and valid. Show inline errors per row before allowing submit.
F.2	**Show loading state**	Same as Mode 1 — spinner, modal stays open.
F.3	**Call Edge Function**	POST /functions/v1/process-single-order. Payload: { mode: 'client', package_code, customers: [{ name, phone, email, delivery_method, save_customer }] }.
F.4	**On success**	Modal transitions to Result Phase. One eSIM card per customer. Toast notification.
F.5	**On partial success**	If some customers succeeded and some failed: modal shows mixed results — green rows for delivered, red rows for failed with reason. Toast: 'X ta eSIM buyurtma qilindi, Y ta xatolik.'

5.2 Backend: process-single-order (mode: client)

#	Step / Action	Detail
B.1	**Auth & partner lookup**	Same as Mode 1.
B.2	**Pre-flight: eSIMAccess balance**	POST /api/v1/open/balance/query. Check balance ≥ (api_price × customer_count). Abort if insufficient.
B.3	**Pre-flight: Eskiz balance**	If any customer has delivery_method = 'sms': GET Eskiz /api/user/get-limit. If balance = 0: switch those customers to Manual delivery, flag 'SMS_BLOCKED_NO_BALANCE', continue.
B.4	**Save customers if flagged**	For each customer where save_customer = true: UPSERT partner_customers (match on partner_id + email or partner_id + phone). Get back customer_id for FK linkage.
B.5	**Loop — for each customer**	Process each customer sequentially (same idempotency pattern as bulk service).
B.6	**Generate transactionId**	Per customer: 'CLIENT-{partner_id_short}-{customer_index}-{timestamp}'.
B.7	**Call eSIMAccess /esim/order**	POST with count: 1. Store orderNo.

B.8	**Poll for allocation**	Same polling logic as Mode 1 — 3s interval, 10 attempts. On timeout: mark customer as 'PENDING_ALLOCATION', include in partial response.
B.9	**Generate token**	nanoid(8), verify uniqueness.
B.10	**Write orders row**	INSERT orders: customer_first_name, customer_last_name, customer_email, customer_phone, end_customer_id (if saved), delivery_method, short_url, all eSIM fields.
B.11	**Write partner_earnings**	One row per successful eSIM.
B.12	**Dispatch delivery**	Based on delivery_method: SMS → Eskiz single send; Email → email provider; Manual → no call, flag order. Log to esim_delivery_logs.
B.13	**Update partner stats**	After loop: aggregate totals, UPDATE partners row.
B.14	**Return response**	Array of per-customer results: { customer_name, order_id, iccid, short_url, qr_code_data, delivery_status, delivery_error }.

5.3 Modal Result Phase

The modal shows a result card per customer. Each card includes:

- Customer name as card header
- Package summary (same for all — one package code for the whole batch)
- Token URL with copy button
- Delivery status chip: 'SMS yuborildi ✓', 'Email yuborildi ✓', or 'Qo'lda tarqatish'
- If failed: red chip with reason, plus 'Qayta urinish' (Retry) button per customer

6. SMS Delivery via Eskiz — Full Integration

Both Mode 1 (qty=1, delivery=SMS) and Mode 2 (per-customer delivery=SMS) use the same Eskiz flow. This section defines it once.

6.1 Auth

#	Step / Action	Detail
E.1	**Login**	POST https://notify.eskiz.uz/api/auth/login. Form fields: email = ESKIZ_EMAIL secret, password = ESKIZ_SECRET_KEY. Both stored as Supabase Edge Function secrets. Response: { data: { token }, token_type: 'bearer' }. Token TTL: 30 days.
E.2	**Store token**	In Edge Function memory only. Never persisted to DB. On each invocation, get a fresh token. (Optimization post-MVP: cache with expiry check using PATCH /api/auth/refresh.)
E.3	**Balance check**	GET https://notify.eskiz.uz/api/user/get-limit with Authorization: Bearer {token}. Check response.data.balance > 0. If zero: abort SMS delivery for all affected orders, log ESKIZ_BALANCE_ZERO event, continue to return result.

6.2 Single SMS Send (Mode 1, qty=1)

#	Step / Action	Detail
S.1	**Compose SMS body**	Template: '{CompanyName}: eSIM tayyor! {Country} {DataGB}GB. O'rnatish: {short_url}'. company_name = partners.company_name. Country and DataGB from esim_packages. Keep under 70 chars for single-part GSM.
S.2	**Call Eskiz send**	POST /api/message/sms/send. Form fields: mobile_phone = phone number (numeric, no +, e.g. 998901234567), message = sms_body, from = '4546', callback_url = 'https://{project}.supabase.co/functions/v1/eskiz-callback'.
S.3	**Handle response**	Response: { id: request_uuid, status: 'waiting' }. Store request_uuid in esim_delivery_logs.provider_message_id. Write delivery log: status = 'sent', sent_at = now(), method = 'sms', recipient_contact = phone (last 4 digits visible only for privacy).
S.4	**Update order**	UPDATE orders.delivery_status JSONB: { method: 'sms', status: 'sent', sent_at, provider_message_id }.

6.3 Batch SMS Send (Mode 2, multiple customers with SMS)

#	Step / Action	Detail
B.1	**Build messages array**	For all customers in this order with delivery_method = 'sms': construct array: [{ user_sms_id: order.id, to: phone_number_integer, text: per_customer_sms_body }]. Each customer's SMS body uses their own short_url token.

B.2	**Call Eskiz send-batch**	POST /api/message/sms/send-batch. Body JSON: { messages: [...], from: '4546', dispatch_id: numeric_hash_of_batch_identifier, callback_url: eskiz_callback_url }.
B.3	**Handle batch response**	Response: { id: batch_request_uuid, status: ['waiting', 'waiting', ...] }. The batch id is one UUID for the entire batch. Per-message tracking is via user_sms_id (= orders.id) in the callback.
B.4	**Write delivery logs**	One esim_delivery_logs row per customer. All share the same provider_message_id (batch UUID). Individual DLR callbacks will update them separately via eskiz-callback function.

6.4 Eskiz Status Mapping

DELIVRD	Delivered → update esim_delivery_logs.status = 'delivered', delivered_at = status_date
UNDELIV	Undelivered (blacklist / debt / block) → status = 'failed', failure_reason = 'undeliverable'
REJECTD	Rejected (number blacklisted) → status = 'failed', failure_reason = 'rejected_blacklist'
DELETED	Sender error (wrong 'from' value) → status = 'failed', failure_reason = 'invalid_sender'
ACCEPTED	Transmitted to operator, awaiting DLR → keep status = 'sent', no change
ENROUTE	In transit → keep status = 'sent', no change
EXPIRED	TTL expired before delivery → status = 'failed', failure_reason = 'sms_expired'
UNKNOWN	Unknown → log but no state change. Flag for review.

eskiz-callback Edge Function

URL: https://{project}.supabase.co/functions/v1/eskiz-callback (public, no auth required — secured by secret path)

Receives Eskiz POST payload: { request_id, message_id, user_sms_id, phone_number, status, status_date }

Lookup: find esim_delivery_logs where provider_message_id = request_id AND order_id = user_sms_id

Apply status mapping above. UPDATE delivery log and orders.delivery_status JSONB.

Log each callback event to payment_audit_log for full audit trail.

7. eSIMAccess API — Specific Implementation Notes

7.1 Authentication

All calls use header: RT-AccessCode: {ESIMACCESS_ACCESS_CODE} (stored as Supabase secret). For production, upgrade to HMAC-SHA256 signature for stronger security — required headers: RT-AccessCode, RT-RequestID (UUID v4), RT-Timestamp (Unix ms), RT-Signature (HMAC-SHA256 of timestamp + requestId + accessCode + body).

7.2 Order Call

Endpoint	POST https://api.esimaccess.com/api/v1/open/esim/order
transactionId	Required. Our idempotency key. Format: '{MODE}-{partner_id_8chars}-{uuid4_short}'. Store BEFORE calling API. If API call is retried, same transactionId means eSIMAccess will not double-charge.
count	Mode 1: quantity from input. Mode 2: always 1 (called per customer). Max 30 per batch.
price validation	Include price and amount fields from esim_packages.api_price. This acts as a price consistency check — eSIMAccess rejects if prices drift.
Response: orderNo	Batch identifier. Store in orders.order_no (maps to orders.esim_tran_no). Required for polling.

7.3 Polling for Allocation

eSIMAccess processes asynchronously. After order, you must retrieve the eSIM data:

- POST /api/v1/open/esim/query with orderNo
- Response esimList[n].esimStatus starts as 'GETTING_RESOURCE', transitions to 'GOT_RESOURCE' when ready
- Poll interval: 3 seconds. Max attempts: 10 (30 seconds total). This is the P99 allocation time.
- On GOT_RESOURCE: extract iccid, ac (activation code = LPA string), qrCodeUrl, esimTranNo, shortUrl
- Prefer esimTranNo over iccid — eSIMAccess documentation warns ICCIDs can be reused across different profiles
- If still not ready after 10 polls: return orderNo to frontend, frontend shows 'eSIM tayyor bo'lmoqda' (eSIM being prepared) with auto-refresh

7.4 eSIM Data Fields to Store

iccid	orders.iccid — eSIM ICCID
esimTranNo	orders.esim_tran_no — Primary reference for all future API calls (suspend, cancel, usage)
ac	orders.activation_code — Full LPA string: 'LPA:1$smdp.address.com$MATCHINGID'. Also used to generate QR client-side.

qrCodeUrl	orders.qr_code_url — Hosted QR image from eSIMAccess CDN (static.redteago.com)
shortUrl	eSIMAccess also provides their own shortUrl — do NOT use this as our token URL. Use our generated onesim.uz/e/{token} instead. Their URL bypasses our tracking.
smdpStatus	orders.smdp_status — Track SM-DP+ server status for support troubleshooting
expiredTime	orders.expiry_date — When the eSIM data plan expires

7.5 Pre-flight: Balance Check

Before any eSIMAccess order call, check account balance:

- POST /api/v1/open/balance/query — empty body
- Response obj.balance is in units×10000 (940000 = $94.00)
- Compare: balance ≥ (package.api_price × quantity)
- package.api_price is already stored in esim_packages.api_price (same units×10000 format)
- If insufficient: return error immediately. Do NOT call /esim/order. Show partner: 'Hisobda mablag' yetarli emas. ONESIM bilan bog'laning.'

8. Database Writes — What Gets Written, When

8.1 Mode 1 — Agent Self-Order

Table	Timing	What is written
orders	After GOT_RESOURCE	One row per eSIM. source_type = 'b2b_partner'. end_customer_type = 'b2b_partner_customer'. customer_phone = delivery phone (if SMS). short_url = tokenized link. delivery_method, delivery_status JSONB.
partner_earnings	After each order row	retail_price, partner_paid, discount_amount, discount_rate, status = 'completed'.
esim_delivery_logs	After Eskiz call	One row per SMS send attempt. provider_message_id, status = 'sent', sent_at, method = 'sms'.
payment_audit_log	Start + end	AGENT_ORDER_STARTED, ESIM_PROVISIONED (×N), DELIVERY_SENT (×N), AGENT_ORDER_COMPLETED.
partners	After completion	total_orders++, total_spent+=, total_savings+=, last_order_at.

8.2 Mode 2 — Client Order

Table	Timing	What is written
partner_customers	Before eSIM order	UPSERT only if save_customer = true. Returns customer_id for FK on orders.end_customer_id.
orders	Per customer, after GOT_RESOURCE	customer_first_name, customer_last_name, customer_email, customer_phone, end_customer_id (if saved), delivery_method, short_url.
partner_earnings	Per customer	Same as Mode 1.
esim_delivery_logs	Per delivery attempt	One row per customer per attempt. Method = SMS / email / manual.
payment_audit_log	Per key event	CLIENT_ORDER_STARTED, per-customer ESIM_PROVISIONED, per-customer DELIVERY_SENT, CLIENT_ORDER_COMPLETED.
partners	After completion	Aggregated totals update.

9. Failure Handling

Failure	Detected At	Handling
eSIMAccess balance insufficient	Pre-flight B.2	Abort entire order. Return error to frontend. No API calls made. Show partner error with admin contact.
eSIMAccess order API error	After /esim/order call	Log error to payment_audit_log. If transactionId already used (idempotency): query by transactionId to recover. Otherwise return error.
eSIM not allocated after 30s poll	After 10 polling attempts	Return partial success: { orderNo, status: 'pending_allocation' }. Frontend shows 'eSIM tayyorlanmoqda' with auto-refresh. Background job polls and completes the order.
Eskiz balance = 0	Pre-flight E.3	Skip all SMS delivery for this order. Switch delivery_method to 'manual' in response. Notify admin. Partner sees: 'SMS yuborilmadi — hisobda mablag' yo'q.'
Eskiz API error on send	After Eskiz call	Log failed delivery. eSIM is still provisioned and accessible via token URL. Partner can copy and share link manually. Retry available.
Network timeout to any API	HTTP client timeout (15s)	Log timeout. Return partial result. Mark order for manual review. Do NOT retry automatically on the same function invocation.
Invalid phone format	Frontend validation (F.1)	Blocked before submission. Never reaches backend.
Partner account suspended	B.1 partner status check	Abort immediately. Return 403 with message 'Hisobingiz to'xtatilgan. ONESIM bilan bog'laning.'

10. Edge Functions for This Service

Function	Trigger	Responsibility
process-single-order	HTTP POST (from frontend, authenticated)	Handles Mode 1 and Mode 2 orders. Runs eSIMAccess pre-flight, order, poll, token generation, DB writes, and delivery dispatch. Returns full result.
eskiz-callback	HTTP POST (public — called by Eskiz)	Receives Eskiz DLR callbacks. Maps status codes. Updates esim_delivery_logs and orders.delivery_status.

get-esim-landing	HTTP GET (public)	Serves eSIM data for onesim.uz/e/{token} landing page. Rate limited. Tracks first_viewed_at.
retry-delivery	HTTP POST (manual or cron)	Re-attempts delivery for orders with delivery failed and attempt_count < 3. Shared with Part 2 service.

11. How Part 1 and Part 2 Fit Together

The three modes in this modal are intentionally tiered in complexity:

Mode	Name	Uses	When to use
1	Tur agent nomiga	process-single-order	Quick purchase, partner manages distribution manually via link sharing
2	Mijoz uchun	process-single-order	Named customers, 1–10 people, automated delivery per person
3	Guruh uchun	process-group-order (Part 2)	Pre-built groups of 5–50+ people, full batch pipeline with per-member tracking

The shared infrastructure between both parts:

- Same token format and onesim.uz/e/{token} landing page
- Same esim_delivery_logs table and schema
- Same eskiz-callback Edge Function
- Same get-esim-landing Edge Function
- Same partner_earnings model
- Same Eskiz credentials and auth flow

Mode 3 handoff sequence

1. Partner selects Mode 3 in modal, picks a group, confirms.
2. Frontend calls POST /functions/v1/process-group-order with { group_order_id, package_code, delivery_method }.
3. process-group-order sets group_orders.status = 'api_ordering' and begins the Part 2 pipeline.
4. Frontend immediately shows progress screen: 'Guruh uchun eSIMlar tayyorlanmoqda...' with member count.
5. Frontend polls group_orders.status every 5 seconds until 'completed' or 'failed'.
6. On 'completed': modal shows summary — N delivered, M manual pending. Links to group order detail page.

12. Build Order

Recommended implementation sequence for this service:

1. build process-single-order Edge Function — Mode 1 (agent, qty=1, SMS delivery). This is the smallest complete path.
2. test end-to-end: order → eSIMAccess poll → token → Eskiz SMS → landing page.
3. extend process-single-order for Mode 1 qty > 1 (manual, no SMS, multiple token links in result).
4. add Mode 2 (client) to the same Edge Function — sequential loop, per-customer delivery.
5. build eskiz-callback for DLR receipt tracking.
6. wire Mode 3 tab in frontend — it calls process-group-order (Part 2), not this service.
7. [later] add Click.uz payment step before eSIMAccess order call — placeholder already exists in payments table.

End of Document — Part 1 of 2

See companion document: **Bulk Group Order Service (Part 2)** | ONESIM v1.0 | ONETECH PRO LLC